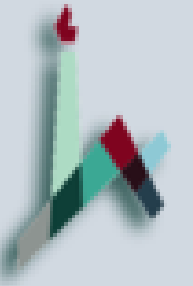


RAG Architecture & Variants

DR. ZVI BEN AMI



RAG Architecture & Variants

AGENDA

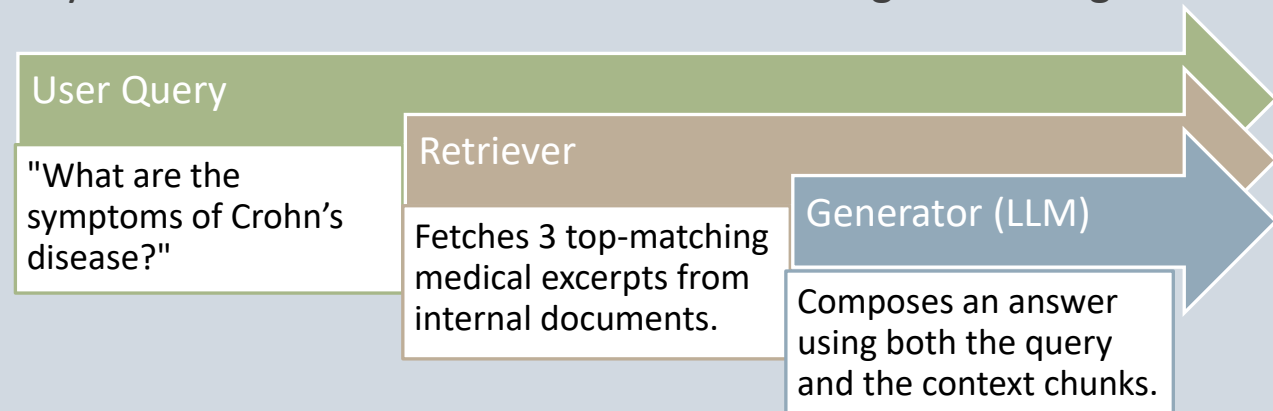
- RAG Motivation & Problem Statement
- Core Pipeline Overview
- Architectural Variants
- Common Pitfalls & Best Practices
- Evaluation & Grounding
- RAG Frameworks & Toolkits
- Summary, Q&A, Resources

LEARNING OBJECTIVES

- By the end of this session, you will be able to:
 - Explain the end-to-end Retrieval-Augmented Generation (RAG) architecture
 - Compare and contrast key RAG variants, including conversational and streaming RAG
 - Evaluate RAG systems using metrics such as Recall@k, MRR, Faithfulness, and RAGAS
 - Recognize and avoid common pitfalls in RAG implementation
 - Apply popular RAG frameworks like LangChain, LlamaIndex, and RAGAS in practice

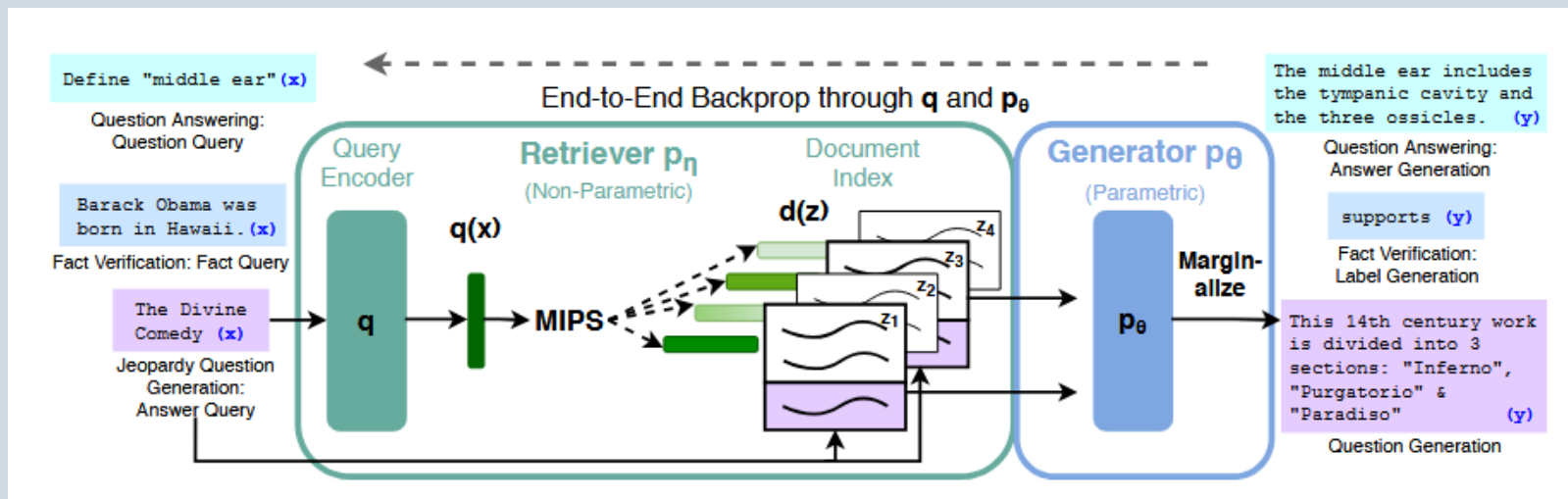
RAG (Retrieval-Augmented Generation)

- RAG is a hybrid architecture that combines two components:
 - Retriever: Fetches relevant documents or chunks from a knowledge base.
 - Generator: Uses a language model (e.g., GPT, BERT) to generate an answer grounded in the retrieved documents.
- Key Components:
 - Retrieval step: Uses semantic search (via vector databases like FAISS or Chroma) to find top-k relevant documents based on the query.
 - Generation step: Passes both the query and retrieved documents to the LLM to generate a grounded, contextualized response.



Meta RAG Architecture Summary

- The RAG architecture, introduced by Facebook AI Research (Lewis et al., 2020) in the paper "Retrieval-Augmented Generation for Knowledge-Intensive NLP".
- It was designed to improve factual accuracy in knowledge-intensive tasks by grounding responses in retrieved documents.
- <https://arxiv.org/pdf/2005.11401>



Why RAG?

- Traditional LLMs are limited by their knowledge cutoff:
 - Once trained, their information is frozen.
 - For example, GPT-3.5's cutoff is September 2021.
- Private & proprietary data:
 - Many enterprise applications require the use of internal or confidential data that LLMs were never trained on.
- Trust & hallucination:
 - Closed LLMs often generate confident but incorrect information. In high-stakes domains like healthcare and law, this erodes trust.

Two Paths to Smarter AI: RAG and Fine-Tuning

- RAG

- Goal:

- Augment a general-purpose LLM with up-to-date, domain-specific knowledge at runtime

- Ideal for:

- Changing or proprietary data
 - Tasks requiring high factual accuracy or auditability
 - Use cases needing real-time or user-specific responses

- Strengths:

- No retraining needed
 - Keeps LLM lightweight and general
 - Easily adaptable to new contexts or user profiles

- Fine-Tuned LLMs

- Goal:

- Adapt the base LLM to perform well on a specific task or domain

- Ideal for:

- Narrow, repetitive tasks (e.g., sentiment analysis, summarization)
 - Use cases where latency or offline capability is critical
 - Scenarios with consistent, static data

- Strengths:

- Lower latency once deployed
 - Can outperform RAG when knowledge doesn't change
 - Easier to deploy on edge devices

	Dynamic Data	Private Knowledge	Offline Use	Static Tasks
RAG	✓	✓	✗	✗
Fine-Tuning	✗	✗	✓	✓

How RAG Solves It?

- Retrieval-Augmented Generation (RAG) dynamically pulls relevant context from external or private sources at runtime, supplementing the LLM with current and domain-specific knowledge.
- Grounded responses:
 - By retrieving from curated documents, RAG reduces hallucination and boosts factual accuracy.
- Secure integration:
 - RAG allows models to access secure internal knowledge bases without retraining.
- Personalization & adaptability:
 - RAG enables injection of task-specific or user-specific context, making the model output more relevant and situationally aware.

RAG Use Cases

- Legal: Access relevant case law and clauses
- Medical: Pull in recent clinical guidelines or lab data
- Customer Support: Retrieve real-time product or troubleshooting docs
- Enterprise: Search across internal wikis or compliance docs

The RAG Flow

Indexing Stage



Runtime Stage



RAG Core Mechanics

- Document Indexing (as previously discussed)

- Chunking Methods

- Fixed-size: Equal-length chunks (e.g., 500 tokens)
 - Overlapping: Adds sliding window context between chunks
 - Semantic (GPT-based): Chunks aligned to logical or semantic boundaries via LLM

- Embedding Models

- MiniLM – Lightweight and fast (384d)
 - E5-base – Instruction-tuned for retrieval and reranking
 - BGE-base – Balanced performance for embedding + cross-encoding

- Vector Storage (as previously discussed)

- FAISS:

- Efficient for approximate nearest-neighbor search
 - Ideal for local use; minimal filtering capabilities
 - ANN library (no DB, you add your own metadata store).
 - Excels at very large corpora and GPU indexing/search

- ChromaDB

- Native support for metadata filtering
 - Designed for LLM/RAG workflows
 - DB features (collections, metadata filters, persistence, easy API).
 - Great up to small–mid corpora

- Indexing Tips

- Use cosine similarity for normalized vectors
 - Attach metadata (e.g., title, section name) for smart filtering
 - Clean and normalize inputs before chunking

RAG Core Mechanics

- Runtime Query Flow
 - Receive user query
 - Embed query using same model as corpus
 - Search vector store for top-k most relevant chunks
 - Combine query + chunks into prompt
 - Pass prompt to LLM → generate grounded answer

- Prompt Formatting (as previously discussed)
 - Prompt Template Example:

```
Context: {chunk_1} {chunk_2}
Question: {user_query}
```

- Best Practices:
 - Use clear instructions: "Answer using only the provided context."
 - Highlight chunk boundaries if needed
 - Control total prompt length to stay within context limits

RAG Architectural Variants

- Different variants of RAG adapt the retrieval-augmented generation pipeline to specific needs such as multi-turn conversations, real-time use, or complex reasoning.
- We will explore:
 - Query Rewrite
 - Improve vague or ambiguous user queries before retrieval to enhance relevance.
 - Conversational RAG
 - Maintain memory across user interactions for context-aware, multi-turn chat.
 - ReAct + RAG
 - Combine reasoning and tool usage with retrieval to support step-by-step thinking.
 - Multi-hop Retrieval
 - Retrieve sequentially across documents to answer complex, layered questions.
 - Multi-vector Retrieval
 - Represent documents using multiple vectors (e.g., title + body) to improve recall.
 - Streaming RAG
 - Generate answers token-by-token in real-time for responsive applications.

[rag_architecture_notebook.ipynb](#)

Query Rewrite

- **What It Is:**

- A technique where an LLM reformulates a vague or ambiguous user query into a more specific one to improve retrieval.
- Not RAG by itself, but a powerful **pre-retrieval** enhancement.
- Boosts **recall**, **relevance**, and **grounding quality** in RAG pipelines.

- **How It Works in RAG:**

- User inputs a query.
- Query is rewritten via an LLM.
- The rewritten query is used for vector similarity search.
- Retrieved chunks are injected into prompt for generation.

- **When to Use:**

- Vague queries (e.g. “What happened with insulin?”)
- Short or context-missing queries
- Conversational agents
- Retrieval failure cases

- **Real-World Use Cases:**

- Enterprise search portals
- Health/Finance bots where users ask incomplete questions
- Legal or compliance document retrieval

- **Latency Impact:**

- Moderate — adds a pre-processing LLM step before retrieval.

- **Cost:**

- Slightly higher due to extra LLM call.

- **Avoid When:**

- Queries are already well-formed and domain-specific.
- Operating under tight latency constraints (e.g., real-time systems).



Query Rewrite - Example

```
import os
from langchain_community.vectorstores import FAISS
from langchain_community.embeddings import HuggingFaceEmbeddings
from langchain.chains import RetrievalQA
from langchain_openai import OpenAI
from langchain.prompts import PromptTemplate
from langchain.schema import Document
```

```
# STEP 1: Define your corpus
documents = [
    "In March 2023, the U.S. government capped insulin prices at $35 for seniors under Medicare Part D.",
    "Insulin price reductions were introduced following years of advocacy for affordable diabetes care.",
    "Crohn's disease is a chronic inflammatory bowel disease that affects the gastrointestinal tract.",
    "Common complications of Crohn's include strictures, fistulas, and malnutrition.",
    "In 2023, Eli Lilly announced a major price drop on several insulin products, including Humalog.",
    "RAG (Retrieval-Augmented Generation) combines language models with document retrieval for better factual grounding.",
    "Patients with Crohn's often require long-term medication and occasional surgery to manage flare-ups.",
    "The Inflation Reduction Act of 2022 contributed to broader changes in drug pricing in the United States.",
    "Conversational RAG keeps memory across user interactions for context-aware answers.",
    "Multi-hop retrieval in RAG helps answer complex queries by combining facts from multiple sources."
]
```

```
# STEP 2: embed and store in FAISS
embedding_fn = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")
vectorstore = FAISS.from_texts(texts=documents,
                                embedding=embedding_fn)
```

```
# STEP 3: Define a query rewriting function
def rewrite_query(original_query: str) -> str:
    if not os.getenv("OPENAI_API_KEY"):
        return original_query # fallback if no API key
    llm = OpenAI(temperature=0.3)
    prompt = PromptTemplate.from_template(
        "Rewrite the following question for better retrieval: {query}"
    )
    return llm(prompt.format(query=original_query))
```

Query Rewrite - Example

```
# STEP 4: Main RAG pipeline
def rag_qa(query: str, k: int = 3) -> str:

    # Retrieve
    docs = vectorstore.similarity_search(query, k=k)
    context = "\n".join(doc.page_content for doc in docs)

    # Generate
    if not os.getenv("OPENAI_API_KEY"):
        print("Retrieved Context:\n", context)
        return "No generation (API key not provided)"

    llm = OpenAI(temperature=0.3)
    prompt = (
        "Given the following context:\n"
        f"{context}\n\n"
        f"Answer the question: {query}"
    )
    return llm(prompt)
```

```
if __name__ == "__main__":
    query = input("Ask a question: ")
    query = rewrite_query(query)
    print(f"🔄 Rewritten Query: {query}")
    answer = rag_qa(query)
    print(f"\n🧠 Answer:\n", answer)
```

Ask a question: "What happened with insulin?"

🔄 Rewritten Query:
"What was the outcome of insulin?"

🧠 Answer:

The outcome of insulin was a reduction in prices, particularly for seniors under Medicare Part D, and a major price drop on several insulin products announced by Eli Lilly.

Conversational RAG

- **What It Is:**

- A Retrieval-Augmented Generation approach designed for multi-turn interactions.
- It maintains a memory of previous user inputs and system responses, enabling more natural, context-aware conversations.

- **How It Works in RAG:**

- Stores prior conversation (e.g., questions & answers) in memory.
- Combines the current query with chat history to perform retrieval.
- Retrieved documents are passed to the LLM along with context for generation.
- Typically uses ConversationBufferMemory or advanced memory objects in LangChain.

- **When to Use:**

- In chatbots or virtual assistants where users ask follow-up questions.
- When context retention across user turns is critical.
- For customer support, medical Q&A, or tutoring systems.

- **Real-World Use Cases:**

- Healthcare assistant: “What’s Crohn’s disease?” → “What are its complications?”
- Legal advisor: “Summarize this law.” → “How does it apply to small businesses?”
- Enterprise search: “Show me our 2023 HR policy.” → “What changed from 2022?”

- **Latency Impact:**

- Moderate to high — conversation history increases context size for retrieval and generation.

- **Cost:**

- Higher due to longer prompts and memory management.

- **Avoid When:**

- Each query is independent (e.g., FAQs or one-shot queries).
- User history is irrelevant or unavailable.

Conversational RAG

```
from langchain.chains import ConversationalRetrievalChain
from langchain.memory import ConversationBufferMemory
from langchain_community.vectorstores import FAISS
from langchain_community.embeddings import HuggingFaceEmbeddings
from langchain_openai import ChatOpenAI
```

```
# —— Setup vector DB ——
documents = [
    "Crohn's disease is a chronic inflammatory bowel disease that affects the gastrointestinal tract.",
    "Common complications of Crohn's include strictures, fistulas, and malnutrition.",
    "Patients with Crohn's often require long-term medication and occasional surgery to manage flare-ups.",
    "Conversational RAG keeps memory across user interactions for context-aware answers."
]
embedding_fn = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")
vectorstore = FAISS.from_texts(texts=documents, embedding=embedding_fn)
```

```
# —— Create Conversational Chain ——
retriever = vectorstore.as_retriever(search_kwargs={"k": 3})
llm = ChatOpenAI(model_name="gpt-3.5-turbo", temperature=0.3)
memory = ConversationBufferMemory(memory_key="chat_history",
return_messages=True)

qa_chain = ConversationalRetrievalChain.from_llm(
    llm=llm,
    retriever=retriever,
    memory=memory)
```

Conversational RAG

```
# —— Multi-turn Chat ——  
query1 = "Tell me about Crohn's disease"  
print("👤", query1)  
print("🤖", qa_chain.run(query1))  
  
query2 = "What are the complications?"  
print("\n👤", query2)  
print("🤖", qa_chain.run(query2))
```

👤 Tell me about Crohn's disease

🤖 Crohn's disease is a chronic inflammatory bowel disease that affects the gastrointestinal tract. It can lead to complications such as strictures, fistulas, and malnutrition. Patients with Crohn's often require long-term medication and occasional surgery to manage flare-ups.

👤 What are the complications?

🤖 Common complications of Crohn's disease include strictures, fistulas, and malnutrition.

User: Tell me about Paris

Assistant: Paris is the capital of France, known for its iconic landmarks such as the Eiffel Tower, the Louvre Museum, and Notre-Dame Cathedral. The city is famous for its art, fashion, cuisine, and romantic ambiance. Paris is also home to numerous cafes, bistros, and patisseries, contributing to its vibrant café culture.

User: What tall structure is there?

Assistant: The tall structure located in Paris is the Eiffel Tower, which stands 324 meters tall.

User: How tall is it?

Assistant: The Eiffel Tower in Paris stands 324 meters tall.

User: Now, what can I see in Rome

Assistant: In Rome, you can see ancient ruins such as the Colosseum and the Roman Forum. These are iconic landmarks that attract many visitors each year.

User: And what about Japan?

Assistant: In Japan, you can visit a variety of attractions such as historic temples, beautiful gardens, traditional tea houses, modern skyscrapers, bustling districts for nightlife and shopping, and much more. Some popular places to visit include Tokyo, Kyoto, Shinjuku, and Shibuya.

ConversationalRAG.py

Conversational RAG - max_token_limit

- **max_token_limit:**
 - caps how many **history** tokens you keep before **auto-summarizing/trimming** (keeps prompts lean).
- Why it matters?
 - Predictable latency/cost
 - Prevents context overflow (retrieved chunks getting pushed out)
 - Enforces a clear truncation policy

```
memory = ConversationSummaryBufferMemory(  
    llm=sum_llm,  
    memory_key="chat_history",  
    return_messages=True,  
    max_token_limit=200,    # small to force summarization for the demo  
)  
  
qa_chain = ConversationalRetrievalChain.from_llm(  
    llm=gen_llm,  
    retriever=retriever,  
    memory=memory,  
    return_source_documents=False,)
```

```
def ask(q: str):  
    print(f"\nUser: {q}")  
    resp = qa_chain.invoke({"question": q})  
    print("Assistant:", resp["answer"])  
    # Show when summarization kicks in  
    summary = getattr(memory, "moving_summary_buffer", "")  
    if summary:  
        print("\n[Memory summary present]")  
        print(summary[:220] + ("..." if len(summary) > 220 else ""))
```

Long-term Conversational Memory

- **Goal:**
 - Enable AI systems to remember facts, preferences, and context across multiple interactions, not just within one session.
- **How it works:**
 1. User shares fact → “My favorite color is blue”.
 2. Embed & store → Save in FAISS / Chroma / DB.
 3. Retrieve later → Match query
 4. What’s my favorite color?” with stored memory.
 5. Inject into LLM → Provide memory as context → Assistant answers “Blue.”

ReAct + RAG

- **What It Is:**

- ReAct (Reasoning + Acting) is a framework that lets language models not only generate answers but also decide when to take actions (e.g., query a retriever) and reason about what to do next.
- Combined with RAG, it empowers models to:
 - Think step-by-step
 - Call tools, like a retriever or calculator
 - Incorporate retrieved knowledge mid-thought

- **How It Works in RAG:**

- The LLM receives a prompt with a question and an action interface (e.g., "Search", "Lookup").
- The LLM decides whether to retrieve, based on reasoning.
- Retrieved docs are fed back into the prompt mid-chain.
- The LLM generates a final answer grounded in external evidence.
- Often implemented using LangChain's AgentExecutor with a retriever tool.

- **When to Use:**

- When queries require tool-use decision-making
- For complex or multi-hop queries (e.g., reasoning across multiple facts)
- When it's unclear whether external retrieval is needed

- **Real-World Use Cases:**

- Financial analyst assistant: "Which insulin companies dropped prices and by how much?"
- Scientific assistant: "Summarize Crohn's treatments introduced in 2023."
- Legal bot: "Was there a new law about insulin pricing last year?"

- **Latency Impact:**

- High — involves iterative reasoning and potentially multiple tool invocations.

- **Cost:**

- High — may invoke the LLM and retriever multiple times.

- **Avoid When:**

- The task is simple (e.g., direct factual questions).
- You can't afford multiple model calls per query.

ReAct + RAG

```
from langchain.agents import Tool, AgentExecutor, initialize_agent
from langchain.agents.agent_types import AgentType
from langchain.tools.retriever import create_retriever_tool
from langchain_community.vectorstores import FAISS
from langchain_community.embeddings import HuggingFaceEmbeddings
from langchain_openai import ChatOpenAI
```

```
# — Document setup —
documents = [
    "In March 2023, the U.S. government capped insulin prices at $35 for seniors under Medicare Part D.",
    "Crohn's disease is a chronic inflammatory bowel disease that affects the gastrointestinal tract.",
    "In 2023, Eli Lilly announced a major price drop on several insulin products, including Humalog.",
    "Common complications of Crohn's include strictures, fistulas, and malnutrition.",
    "RAG (Retrieval-Augmented Generation) combines language models with document retrieval for better factual grounding."
]
```

```
# — Build FAISS retriever —
embedding_fn = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")
vectorstore = FAISS.from_texts(texts=documents, embedding=embedding_fn)
retriever = vectorstore.as_retriever()

# — Create retrieval tool —
retriever_tool = create_retriever_tool(
    retriever=retriever,
    name="insight_search",
    description="Searches the document base for medical and pricing information."
)

# — Initialize ReAct Agent —
llm = ChatOpenAI(temperature=0.3, model_name="gpt-3.5-turbo")
tools = [retriever_tool]

agent = initialize_agent(
    tools=tools,
    llm=llm,
    agent=AgentType.OPENAI_FUNCTIONS,
    verbose=True
)
```

ReAct + RAG

```
# — Example Query —  
query = "What are the new insulin pricing changes announced in 2023?"  
response = agent.run(query)  
print("🤖", response)
```

```
> Entering new AgentExecutor chain...
```

```
Invoking: `insight_search` with `{'query': 'new insulin pricing changes 2023'}`
```

```
In March 2023, the U.S. government capped insulin prices at $35 for seniors under Medicare Part D.
```

```
In 2023, Eli Lilly announced a major price drop on several insulin products, including Humalog.
```

```
Common complications of Crohn's include strictures, fistulas, and malnutrition.
```

```
Crohn's disease is a chronic inflammatory bowel disease that affects the gastrointestinal tract. In  
2023, the U.S. government capped insulin prices at $35 for seniors under Medicare Part D.  
Additionally, Eli Lilly announced a major price drop on several insulin products, including  
Humalog.
```

```
> Finished chain.
```

```
🤖 In 2023, the U.S. government capped insulin prices at $35 for seniors under Medicare Part D.  
Additionally, Eli Lilly announced a major price drop on several insulin products, including Humalog.
```

Multi-hop Retrieval

- **What It Is:**

- A RAG technique where answering a question requires chaining multiple pieces of evidence from different documents.
- It simulates step-by-step reasoning, where the output of one retrieval guides the next.

- **How It Works in RAG:**

- Initial query is used to retrieve relevant documents.
- LLM reads first-hop context, generates follow-up questions or keywords.
- These follow-ups are used to perform a second retrieval.
- The combined context from both hops is passed to the generator for a grounded response.
- Often used in LangChain with MultiVectorRetriever or custom tool chaining.

- **When to Use:**

- When a question requires combining facts from multiple places
- For complex queries: reasoning, comparison, cause/effect
- Ideal for knowledge graphs or large corpora with distributed information

- **Real-World Use Cases:**

- Health: "How did U.S. legislation impact insulin pricing and patient outcomes?"
- Education: "Which Crohn's complications lead to surgery, and what are the outcomes?"
- News: "What events preceded Eli Lilly's price reduction?"

- **Latency Impact:**

- High — requires sequential retrieval passes.

- **Cost:**

- High — multiple retrieval and generation steps.

- **Avoid When:**

- Single-hop queries suffice.
- Latency needs to stay <1s (e.g., voice assistants).

Multi-hop Retrieval

```
from langchain.chains import RetrievalQA
from langchain_openai import ChatOpenAI
from langchain_community.embeddings import HuggingFaceEmbeddings
from langchain_community.vectorstores import FAISS
```

```
# — Sample Corpus —
documents = [
    "In March 2023, the U.S. government capped insulin prices at $35 for seniors under Medicare Part D.",
    "Insulin price reductions were introduced following years of advocacy for affordable diabetes care.",
    "Crohn's disease is a chronic inflammatory bowel disease that affects the gastrointestinal tract.",
    "The Inflation Reduction Act of 2022 contributed to broader changes in drug pricing in the United States.",
    "In 2023, Eli Lilly announced a major price drop on several insulin products, including Humalog.",
    "Patients with Crohn's often require long-term medication and occasional surgery to manage flare-ups."
]
```

```
# — Build FAISS retriever —
embedding_fn = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")
vectorstore = FAISS.from_texts(texts=documents, embedding=embedding_fn)
retriever = vectorstore.as_retriever(search_kwargs={"k": 2})
llm = ChatOpenAI(temperature=0.3, model_name="gpt-3.5-turbo")
```

Multi-hop Retrieval

```
# — Step 1: Initial retrieval and response —
qa_chain = RetrievalQA.from_chain_type(llm=llm, retriever=retriever,
chain_type="stuff")

first_question = "What legislation affected insulin pricing?"
first_answer = qa_chain.run(first_question)
print("Step 1:", first_answer)

# — Step 2: Use LLM output to formulate second-hop query —
follow_up_question = "What did the Inflation Reduction Act do in healthcare?"
second_answer = qa_chain.run(follow_up_question)
print("Step 2:", second_answer)

# — Final summary with both contexts —
final_prompt = f"""
Using this information:
- {first_answer}
- {second_answer}

Summarize how U.S. legislation affected insulin pricing.
"""

final_response = llm.predict(final_prompt)
print("✅ Final Answer:", final_response)
```

Step 1: The U.S. government's decision to cap insulin prices at \$35 for seniors under Medicare Part D in March 2023 was not tied to a specific legislation but rather a regulatory action.

Step 2: The Inflation Reduction Act of 2022 contributed to broader changes in drug pricing in the United States, including introducing insulin price reductions following years of advocacy for affordable diabetes care.

✅ Final Answer: U.S. legislation, such as the Inflation Reduction Act of 2022, played a significant role in influencing insulin pricing by introducing measures to reduce drug costs, including insulin. Additionally, the government's regulatory action to cap insulin prices at \$35 for seniors under Medicare Part D in March 2023 further impacted insulin pricing without being tied to a specific legislation. These legislative and regulatory actions aimed to make insulin more affordable for individuals in need of diabetes care.

Multi-vector Retrieval

- **What It Is:**

- A technique where each document or passage is represented by multiple dense vectors, capturing different semantic aspects or subcomponents (e.g., title, body, metadata).
- This improves retrieval recall and relevance, especially for complex or multi-faceted queries.

- **How It Works in RAG:**

- Each document is split into logical parts (e.g., title, paragraph, summary).
- Each part is embedded separately into its own vector.
- During retrieval, the query is matched against all vectors, and top-scoring matches are used to reconstruct document context.
- Typically used with vector stores that support multi-field retrieval or hybrid indexes.

- **When to Use:**

- When documents have multiple relevant fields (e.g., FAQ title + answer)
- When retrieval needs to match varied query phrasings
- For complex document types (e.g., reports, EHRs, scientific papers)

- **Real-World Use Cases:**

- Health: Matching diagnosis queries against symptoms + clinical notes
- Customer support: Matching “How do I refund?” against title + body of help articles
- Enterprise search: Searching based on document metadata + full content

- **Latency Impact:**

- Slightly higher — more embeddings per document to search over.

- **Cost:**

- Higher upfront (indexing), but manageable at inference.

- **Avoid When:**

- Corpus is simple or short-form (e.g., short notes).
- Storage space is limited (index size grows with multi-vectoring).

Multi-vector Retrieval

```
from langchain_community.embeddings import HuggingFaceEmbeddings
from langchain_community.vectorstores import FAISS
from langchain.chains import RetrievalQA
from langchain_openai import ChatOpenAI

# — Define multi-part documents —
documents = [
    {"title": "Insulin Price Cap",
     "body": "In March 2023, the U.S. government capped insulin prices at $35 for Medicare seniors."},
    {"title": "Crohn's Complications",
     "body": "Common complications of Crohn's include strictures, fistulas, and malnutrition."},
    {"title": "Eli Lilly Insulin", "body": "Eli Lilly reduced prices of insulin in 2023, including Humalog."}
]
```

```
# — Flatten documents into multiple embeddings per field —
texts = []
metadatas = []

for doc in documents:
    texts.append(doc["title"])
    metadatas.append({"source": "title"})

    texts.append(doc["body"])
    metadatas.append({"source": "body"})

# — Create vector store —
embedding_fn = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")
vectorstore = FAISS.from_texts(texts=texts, embedding=embedding_fn,
                               metadatas=metadatas)

# — Set up retriever and chain —
retriever = vectorstore.as_retriever(search_kwargs={"k": 4})
llm = ChatOpenAI(temperature=0.3, model_name="gpt-3.5-turbo")
qa_chain = RetrievalQA.from_chain_type(llm=llm, retriever=retriever)
```

Multi-vector Retrieval

```
# — Ask complex query that benefits from title+body  
match —  
query = "Which company lowered insulin prices and why?"  
response = qa_chain.run(query)  
  
print("✅ Final Answer:", response)
```

```
✅ Final Answer: Eli Lilly lowered insulin prices in 2023, including Humalog, as  
part of an effort to make insulin more affordable for patients.
```

Streaming RAG

- **What It Is:**

- Streaming RAG combines retrieval-augmented generation with token-by-token streaming output from the LLM.
- Rather than waiting for the full response, the model begins generating output as soon as it retrieves relevant context — improving responsiveness and enabling real-time UX.

- **How It Works in RAG:**

- Retrieval is done before generation as in standard RAG.
- A streaming-capable LLM (e.g., OpenAI's stream=True) begins generating output incrementally.
- Can be integrated into UIs like chatbots, notebooks, or real-time dashboards.

- **When to Use:**

- Real-time interactive applications (e.g., chat, copilots)
- Long-form answers where early visibility helps the user
- Progressive rendering systems (e.g., Web apps with token-streaming)

- **Real-World Use Cases:**

- Customer service chatbots with instant feedback
- Medical assistants summarizing clinical history while retrieving new documents
- Legal research tools drafting memos with streaming output

- **Latency Impact:**

- Low perceived latency — starts responding immediately.

- **Cost:**

- Similar to regular generation, but with streaming infra setup.

- **Avoid When:**

- Output quality must be fully validated before display.
- No streaming UI/UX exists (e.g., static reports).

Streaming RAG

```
from langchain_community.embeddings import HuggingFaceEmbeddings
from langchain_community.vectorstores import FAISS
from langchain_community.chat_models import ChatOpenAI
from langchain.chains import RetrievalQA
from langchain.callbacks.streaming_stdout import StreamingStdOutCallbackHandler

# — Sample corpus —
documents = [
    "In March 2023, the U.S. government capped insulin prices at $35 for seniors under Medicare Part D.",
    "Crohn's disease is a chronic inflammatory bowel disease that affects the gastrointestinal tract.",
    "RAG (Retrieval-Augmented Generation) combines language models with document retrieval for better factual grounding.",
    "Streaming RAG enables real-time, token-by-token responses from language models.",
    "Multi-hop retrieval in RAG helps answer complex queries by combining facts from multiple sources."
]
```

```
# — Build vector DB —
embedding_fn = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")
vectorstore = FAISS.from_texts(texts=documents, embedding=embedding_fn)

# — Set up retriever and LLM with streaming —
retriever = vectorstore.as_retriever()
streaming_llm = ChatOpenAI(
    temperature=0.2,
    streaming=True,
    callbacks=[StreamingStdOutCallbackHandler()],
    model_name="gpt-3.5-turbo"
)

qa_chain = RetrievalQA.from_chain_type(llm=streaming_llm, retriever=retriever)

# — Run a query (streaming output will print token-by-token) —
query = "What is Streaming RAG and why is it useful?"
print("👤", query)
print("🤖", end=" ")
qa_chain.run(query)
```

👤 What is Streaming RAG and why is it useful?

🤖 Streaming RAG enables real-time, token-by-token responses from language models. It is useful because it allows for immediate and continuous generation of responses as the input is being processed, which can be beneficial for applications requiring real-time interactions or dynamic content generation.

Variants Comparison

Variant	 Latency Impact	 Cost	 When to Use	 Avoid When
Query Rewrite	 Medium	 Medium	When queries are vague, short, or conversational; useful in chatbots or voice assistants	When queries are already precise; tight latency constraints
Conversational	 High	  High	When prior turns matter (e.g., chatbots, virtual assistants, tutoring, medical Q&A)	For one-off queries; when context history isn't important
ReAct + RAG	 High	   Very High	When reasoning is required before deciding to retrieve (e.g., multi-step decision workflows)	For factual or short queries; cost- or latency-sensitive cases
Multi-hop	 High	  High	When chaining multiple facts is required (e.g., cause-effect, legislation impact analysis)	When a single retrieval hop is enough; voice-based systems
Multi-vector	 Medium	  Medium-High	When documents contain multiple fields (e.g., title/body); improves recall across query types	When docs are short/simple; if index storage is a constraint
Streaming RAG	 Low	 Medium	When you need fast user feedback (e.g., real-time chat, dashboards, live notebooks)	When output must be fully verified before display

Which RAG variant best fits your use case?

RAG Evaluation

- RAG systems retrieve external knowledge and generate responses — both steps can introduce errors.
- Evaluation ensures the retrieved context is relevant and the generated answer is faithful to that context.
- Two Pillars of Evaluation:
 - Retrieval Evaluation
 - Recall@k: Are the right documents retrieved?
 - Context relevance: Are they useful for answering the question?
 - Generation Evaluation
 - Faithfulness: Is the answer grounded in the retrieved context?
 - Fluency & Relevance: Is the answer correct, complete, and well-formed?
- Evaluation Techniques:
 - Automated metrics: Precision, Recall, MRR, etc.
 - LLM-based scoring: Using models to assess grounding and factuality.
 - Manual spot-checking for high-stakes domains.

RAG Faithfulness

- Faithfulness measures how well the generated answer is grounded in the retrieved context.
 - A faithful answer does not introduce unsupported facts.
- Methods of Evaluation:
 - String-based Matching (Basic):
 - Check whether key facts in the generated answer appear verbatim or with slight variation in the retrieved chunks.
 - Low recall for paraphrased facts.
 - LLM-based Evaluation (Preferred):
 - Uses a language model (e.g., GPT) to assess whether the generated answer is fully supported by the retrieved context.
 - Can understand paraphrases and latent meaning.
- Faithfulness
 - For each question:
 - Compare answer $\leftrightarrow$ contexts
 - If answer includes only information found in contexts, mark as faithful (score = 1).
 - If answer includes hallucinated or unsupported claims, mark as unfaithful, mark as unfaithful (score = 0).

$$\text{Faithfulness} = \frac{\text{Total Number of Answers}}{\text{Number of Faithful Answers}}$$

RAGAS

- RAGAS (Retrieval-Augmented Generation Assessment Suite)
- An open-source framework developed by Exploding Gradients for evaluating RAG pipelines using multiple metrics beyond retrieval accuracy.
- Key Evaluation Metrics:
 - Faithfulness – Is the answer grounded in retrieved content?
 - Answer Relevancy – Does the answer address the question?
 - Context Precision – Are the retrieved contexts useful and not redundant?
 - Retrieval Recall – Are the relevant documents included in the top-k?
- How it works:
 - Compare the generated answer against the ground truth and retrieved contexts.
 - Uses a mixture of string metrics and LLM-based evaluation.
- Install
 - `pip install ragas`

RAGAS Evaluation Example

```
from ragas.metrics import faithfulness, answer_relevancy
from ragas.evaluation import evaluate
from ragas.testset import Testset
from datasets import Dataset

# ✅ Create samples as dictionaries
samples = [
    {
        "question": "What is Crohn's disease?",
        "answer": "Crohn's disease is a chronic inflammatory bowel disease that affects the gastrointestinal tract.",
        "ground_truth": "Crohn's disease is a chronic inflammatory bowel disease that affects the gastrointestinal tract.",
        "contexts": ["Crohn's disease is a chronic inflammatory bowel disease that affects the gastrointestinal tract.",
                    "Patients with Crohn's often require long-term medication and occasional surgery to manage flare-ups."]
    },
    {
        "question": "How did the Inflation Reduction Act affect drug pricing?",
        "answer": "The Inflation Reduction Act of 2022 contributed to broader changes in drug pricing in the United States.",
        "ground_truth": "The Inflation Reduction Act of 2022 contributed to broader changes in drug pricing in the United States.",
        "contexts": ["The Inflation Reduction Act of 2022 contributed to broader changes in drug pricing in the United States.",
                    "In March 2023, the U.S. government capped insulin prices at $35 for seniors under Medicare Part D."]
    }
]
```

```
# ✅ Convert to HuggingFace Dataset format (required by RAGAS)
dataset = Dataset.from_list(samples)

# ✅ Evaluate using the dataset directly
results = evaluate(
    dataset,
    metrics=[faithfulness, answer_relevancy]
)

# ✅ Output results
print("📊 RAGAS Evaluation Metrics:")
print(results)
```

```
📊 RAGAS Evaluation Metrics:
{'faithfulness': 1.0000, 'answer_relevancy': 0.9854}
```

RAG Evaluation Summary

Metric	Failure Mode
Faithfulness	Model cites facts not supported by retrieved context (hallucination).
Context Precision	Retrieved too much irrelevant context (prompt flooding).
Context Recall	Missed key facts; chunking or retriever configuration needs tuning.
Answer Relevancy	Answer doesn't address the question; prompts/instructions are misaligned.
Answer Correctness	Mismatch vs. ground truth; retrieval/reranking or source quality may be weak.

Evaluate using LLM as a Judge

- What it is?
 - Use a second LLM to score a model's answer against the provided CONTEXT, returning structured metrics (e.g., Groundedness & Factuality on 1–5).
 - Think of it as an automated, rubric-driven reviewer.
- Why use it?
 - Scales human-style review to many Q/A pairs
 - Catches hallucinations (low groundedness) and mistakes (low factuality)
 - Produces consistent, machine-readable scores + evidence
- How it works?
 1. Generate answer using a “use context only” prompt
 2. Judge (using the same or different model) with a fixed rubric → `judge_chain.invoke({context, question, answer})`
 3. Log JSON scores + evidence for dashboards/alerts

Evaluate using LLM as a Judge

```
# ----- Structured schema -----
class GroundingJudgment(BaseModel):
    grounded_score: int = Field(ge=1, le=5, description="5 = fully
grounded in context")
    grounded: bool
    factual_score: int = Field(ge=1, le=5, description="5 =
factually correct")
    factual: bool
    evidence_quotes: List[str] = Field(description="Direct quotes
from CONTEXT supporting the answer")
    missing_evidence_notes: str = Field(description="What evidence
is missing in CONTEXT, if any")
    reasoning: str = Field(description="Brief rationale for both
scores")
```

```
# ----- Prompt -----
rubric = """
You are a strict evaluator of a QA system. Judge ONLY the provided data.

Scoring rules:
- Groundedness (1-5): Is the answer fully supported by the CONTEXT?
  5 = fully supported with explicit matching evidence
  3 = partially supported / requires inference beyond context
  1 = unsupported or contradicted by context
- Factuality (1-5): Is the answer factually correct given the CONTEXT and
widely-accepted knowledge?
  5 = correct; 3 = partly correct or incomplete; 1 = incorrect

Return JSON ONLY matching the schema.
"""

prompt = ChatPromptTemplate.from_messages([
    ("system", rubric),
    ("human",
     "CONTEXT: \n{context}\n\nQUESTION: \n{question}\n\nANSWER: \n{answer}\n\n"
     "Produce a strict JSON object adhering to the provided schema.")])

llm = ChatOpenAI(model="gpt-4o-mini", temperature=0) # pick your model

# Bind the output to the Pydantic schema:
judge_chain = prompt | llm.with_structured_output(GroundingJudgment)
```

Evaluate using LLM as a Judge

```
# ----- Example usage -----
context = (
    "Infliximab is an anti-TNF biologic used for moderate-to-severe Crohn's disease. "
    "First-line therapy may include corticosteroids for induction; maintenance can use biologics."
)
question = "What is a first-line therapy for moderate-to-severe Crohn's disease?"
answer = "Infliximab is commonly used; steroids can induce remission, biologics maintain it."

result: GroundingJudgment = judge_chain.invoke(
    {"context": context, "question": question, "answer": answer}
)
print('Judge evaluation:', result)
```

```
Judge evaluation: grounded_score=5 grounded=True factual_score=5 factual=True
evidence_quotes=["Infliximab is an anti-TNF biologic used for moderate-to-severe Crohn's disease.", 'First-line therapy may include corticosteroids for induction; maintenance can use biologics.'] missing_evidence_notes='' reasoning='The answer is fully supported by the context, which explicitly states that corticosteroids can be used for induction and biologics like Infliximab are used for maintenance. The answer accurately reflects this information.'
```


End-to-End RAG Demo (Photography Domain)

- **Build ([build_rag.py](#))**
 - Download Wikipedia pages on photography topics (aperture, shutter, ISO, etc.)
 - Chunk text → embed with text-embedding-3-small
 - Store in Chroma vector DB (persist locally)
 - Smoke test: simple Q&A to verify index works
- **Evaluate ([eval_rag.py](#))**
 - Custom metrics:
 - KeywordRecall (expected terms in answer)
 - CitationRecall (expected docs cited)
 - Cited $\in$ Retrieved (cited docs in top-k)
 - RAGAS metrics:
 - Answer Relevancy, Faithfulness
 - Context Precision, Context Recall
- **Chat ([chat_rag.py](#))**
 - Wrap retriever as a LangChain Tool
 - Use ChatOpenAI + AgentType.OPENAI_FUNCTIONS
 - Agent always: *retrieve* → *ground* → *generate with citations*
 - Demo queries:
 - “What settings control exposure?”
 - “How does aperture affect depth of field?”
 - “What is the rule of thirds?”
 - ”Structured JSON: {setting, effect, creative_use}

Production Considerations

- Building a reliable RAG system in production requires architectural decisions that balance speed, relevance, and scalability.
- Caching Retrievals
 - Reduces latency and cost, especially for frequently asked queries.
 - Use an LRU cache or Redis to store recent query → top-k results.
 - Cache both retrieval outputs and final generated answers if stable.
 - Normalize queries (e.g., lowercase, remove punctuation) to improve hit rate.
- Dynamic Chunk Sizing
 - One-size-fits-all chunking can either lose context or exceed token limits.
 - Dynamically adjust chunk size based on document structure or query specificity.
 - Use heading-based or semantic chunking for long documents.
 - Apply recursive splitting for oversize content (as previously discussed).
- Metadata-Based Filtering
 - Improve retrieval precision in large, heterogeneous corpora.
 - Only search within `document_type="clinical"` or `department="legal"`
 - Restrict by `date_range`, `author`, `version`, etc.
- Best Practice
 - Combine semantic retrieval + metadata filters for hybrid search.
 - Monitor query latency and cache hit rate continuously

Do & Don't

- **Common Pitfalls**

- Over-retrieval (prompt flooding)
- Redundant or conflicting documents
- Poor chunking leading to low signal

- **Best Practices**

- Test with different chunk sizes
- Evaluate with real examples
- Balance between recall and faithfulness

- **Evaluation & Grounding**

- Retrieval quality directly impacts answer quality
- Faithfulness vs. fluency tradeoffs

- **Metrics Table**

- Recall@k: Was the correct doc in top-k?
- MRR: Mean Reciprocal Rank (how early we found it)
- RAGAS: holistic evaluation (faithfulness, answer correctness)

Frameworks & Tools

- LangChain

- Website: <https://www.langchain.com>
- Popular Chains:
 - RetrievalQA: <https://docs.langchain.com/docs/modules/chains/popular/retrieval-qa>
 - ConversationalRetrievalChain: https://python.langchain.com/docs/use_cases/question_answering/chat_vector_db
- Core Tools:
 - PromptTemplate: <https://docs.langchain.com/docs/modules/prompts/prompt-templates/>
 - OutputParser: https://docs.langchain.com/docs/modules/model_io/output_parsers/

- LlamaIndex

- Website: <https://www.llamaindex.ai>
- Index Types:
 - VectorIndex: https://docs.llamaindex.ai/en/stable/examples/vector_stores/
 - SummaryIndex: <https://docs.llamaindex.ai/en/stable/examples/indices/summary/>
 - TreeIndex: <https://docs.llamaindex.ai/en/stable/examples/indices/tree/>
 - ListIndex: <https://docs.llamaindex.ai/en/stable/examples/indices/list/>
- Key Components:
 - ServiceContext: https://docs.llamaindex.ai/en/stable/api_reference/service_context/
 - QueryEngine: https://docs.llamaindex.ai/en/stable/api_reference/query_engine/

- Other Tools

- Haystack: <https://haystack.deepset.ai>
- LangSmith: <https://www.langchain.com/langsmith>
- DSPy: <https://github.com/stanford-crfm/dspy>
- RAGflow: <https://github.com/IBM/RAGFlow>

Further Resources

- **Documentation & Guides**

- LangChain Docs: <https://docs.langchain.com/>
- LlamaIndex Docs: <https://docs.llamaindex.ai/>
- Haystack Docs: <https://docs.haystack.deepset.ai/>
- LangSmith (LangChain Observability & Evaluation): <https://www.langchain.com/langsmith>

- **Repositories**

- LangChain GitHub: <https://github.com/langchain-ai/langchain>
- LlamaIndex GitHub: https://github.com/jerryliu/llama_index
- Haystack GitHub: <https://github.com/deepset-ai/haystack>
- RAGAS (Evaluation Toolkit): <https://github.com/explodinggradients/ragas>
- DSPy (Stanford - LLM Programming): <https://github.com/stanford-crfm/dspy>

- **Tutorials & Articles**

- Lewis et al., 2020 – RAG Paper: <https://arxiv.org/abs/2005.11401>
- LangChain – Chat with Your Data Tutorial: <https://docs.langchain.com/docs/use-cases/chat-with-your-data>
- LlamaIndex – RAG Example: <https://docs.llamaindex.ai/en/stable/examples/retrieval/>
- ReAct: Synergizing Reasoning and Acting in Language Models: <https://arxiv.org/abs/2210.03629>
- Top 10 LLM & RAG Projects for Your AI Portfolio (2025–26): <https://techwithram.medium.com/top-10-llm-rag-projects-for-your-ai-portfolio-2025-26-582cc7ab6507>

Wrap-Up & Q&A

- **Key Takeaways:**

- Retrieval-Augmented Generation (RAG) enhances LLMs by grounding their outputs in external, up-to-date, or domain-specific knowledge.
- Core RAG pipeline consists of two stages: retrieving relevant documents and generating a grounded response.
- Variants of RAG (e.g., Conversational RAG, ReAct, Multi-hop, Streaming) expand its capabilities for specific use cases such as chatbots, reasoning, and real-time interaction.
- Evaluation matters — metrics like Recall@k, MRR, Faithfulness, and RAGAS ensure system quality and trust.
- Frameworks like LangChain and LlamaIndex simplify RAG implementation and offer robust tooling for real-world deployment.
- Common pitfalls include poor chunking, redundant retrieval, and lack of evaluation — these can be avoided with best practices in chunking, caching, and hybrid filtering.

Wrap-Up & Q&A

- **Next Steps**

- Apply what you've learned:
 - Build a simple RAG pipeline using LangChain or LlamaIndex.
 - Experiment with different chunking and embedding strategies.
 - Try a RAG variant like Conversational RAG or Streaming RAG using example notebooks.
- Evaluate your pipeline:
 - Use RAGAS or manual checks to assess answer grounding and retrieval quality.
 - Tune your chunk sizes and retrieval top-k to balance performance and cost.
- Explore further tools:
 - Try LangSmith for tracing and debugging.
 - Explore DSPy for programmable LLM pipelines.
- Prepare for what's next:
 - Bring your own use case or idea to experiment with RAG or agent-based workflows.

Upcoming Meetings

- Module 25: LLM Methodology & Ecosystem (16.09 – Frontal)
- Module 26: LLM as an instrument - Data cleansing - LLM ready (18.09 – Zoom)
- Module 27: Embedding Strategies, Retrieval Pipelines, & Vector Stores (25.09 – Zoom)
- Module 28: RAG Architecture & Variants (30.09 – Zoom)
- **Module 29: AI Agents (16.10 – Zoom)**
- **Module 30: MCP Architecture (21.10 – Zoom)**

THANK YOU FOR LISTENING

ZVI.BENAMI@MAIL.HUJI.AC.IL

